



Actividad práctica en parejas: legajos únicos y cupos que se respetan

Objetivo: ver fallar (y arreglar) la diferencia entre miembros de instancia y miembros de clase, y usar el paso de referencias para que dos clases colaboren sin romper el encapsulamiento.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya lo clonaron en la clase de Git, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es 04-colaboracion-y-static/actividad, y adentro hay tres clases:

Estudiante.java y Curso.java son los archivos que hay que modificar.

GestionAcademica.java es el programa que los usa. No se modifica, salvo para comentar la parte 1 y descomentar la parte 3 cuando el enunciado lo indique.

Parte 1: ver el problema

Ejecutar GestionAcademica tal como viene y mirar la salida. Responder entre los dos, por escrito y en pocas palabras:

¿Qué legajo recibió cada estudiante? El constructor tiene la línea que asigna y después incrementa, y esa línea parece correcta. Si el error no está en esa línea, ¿dónde está?

El curso se creó con cupo para 2 y terminó con 4 inscriptos. El programa no falló ni avisó nada. ¿Quién tendría que haberlo impedido: el main que inscribe, o el curso?

Escribir en una frase la promesa de Estudiante sobre los legajos, y en otra la promesa de Curso sobre el cupo.

Parte 2: arreglar las dos clases

Los pasos están marcados como TODO dentro de los archivos. En orden:

TODO 1 (Estudiante): convertir proximoLegajo en un miembro de clase. Es un cambio de una palabra; lo importante es poder explicar por qué esa palabra arregla los cuatro legajos.

TODO 2 (Estudiante): el constructor valida nombre y apellido y corta con throw new IllegalArgumentException("...") si no sirven, igual que la clase pasada. El objeto se niega a nacer inválido.



TODO 3 (Estudiante): escribir `getProximoLegajo()`. La decisión importante no es el cuerpo del método, que tiene una línea: es si el método es de una instancia o de la clase. Pregunta guía: ¿podría usar `this` adentro? ¿Sobre qué objeto estaría parado?

TODO 4 (Curso): `inscribirEstudiante` hace cumplir la promesa del curso: rechaza `null`, rechaza al que ya está inscripto y rechaza cuando no queda cupo. Convención de la cátedra, la misma de los setters: el método no imprime nada, devuelve `false`, y quien llama decide cómo avisar.

Una nota sobre el duplicado: `this.inscriptos.contains(unEstudiante)` compara referencias. La pregunta que responde es si la lista ya guarda a ese mismo objeto, no a uno parecido. Que eso alcance para detectar la reinscripción dice algo sobre qué guarda realmente la lista, y es el tema de la parte 3.

Parte 3: verificar y discutir

Comentar la parte 1 de `GestionAcademica` y descomentar la parte 3. La salida esperada, en orden: legajos correlativos desde 10001; dos inscripciones aceptadas y la tercera rechazada; la reinscripción de Houssay rechazada; el cambio de apellido hecho desde `e1` apareciendo en el listado del curso; el `int` que no cambia después de `intentarModificar`; el próximo legajo consultado con el nombre de la clase; y el programa cortándose con el mensaje del constructor en la última línea. Que termine con esa excepción es el resultado correcto.

Para discutir entre los dos antes del cierre:

El apellido se cambió con `e1.cambiarApellido(...)`, sin tocar el curso, y el listado del curso lo mostró cambiado. ¿Cuántos objetos `Estudiante` de Houssay hay en memoria? ¿Qué guarda exactamente la lista `inscriptos`?

¿Por qué `intentarModificar` no pudo cambiar `cupoLocal`, si `cambiarApellido` sí cambió lo que muestra el curso? ¿Qué se copió en cada caso?

`Estudiante.getProximoLegajo()` se invoca con el nombre de la clase, sin ningún objeto. `e1.getProximoLegajo()` también compila. ¿Por qué está mal escribirlo así, si funciona?

En el proyecto integrador esto aparece en todos lados: cada vez que una clase guarde una lista de otra, o reciba un objeto por parámetro, van a estar usando exactamente lo de hoy.